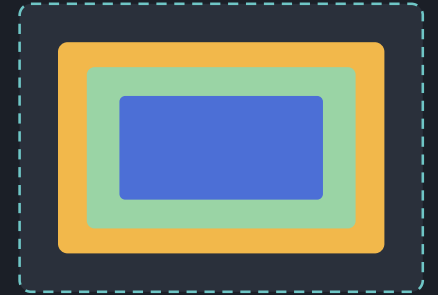


LECTURE 08 · WEB DEVELOPMENT COURSE

The CSS Box Model

Every element on a page is a box. Today we learn exactly what's inside that box.

`content/padding/border/margin``box-sizing``margin collapsing``display property`

Roadmap

Today's Roadmap

- 01 Every element is a box — the four components
- 02 Content, padding, border, margin — one at a time
- 03 box-sizing: content-box vs border-box
- 04 Margin collapsing — the gap that isn't what you expect
- 05 The display property: block, inline, inline-block, none

The big idea

Every Element Is a Box

Whether it's a `<p>`, a `<button>`, or a whole `<div>`, the browser treats every single HTML element as a rectangular box — even if it doesn't look like one.

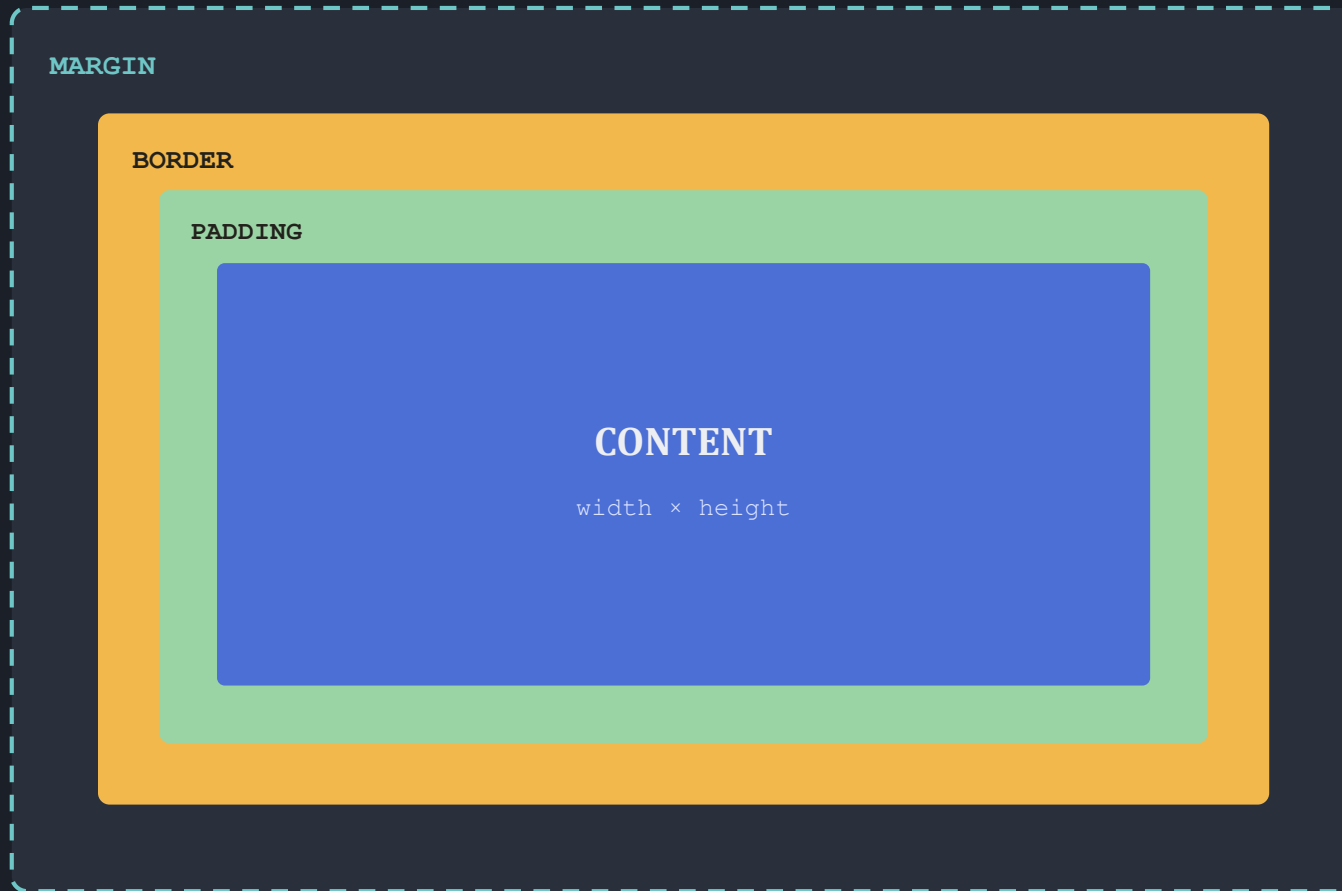
The box model is the set of rules for how big that box actually is, once you add spacing and borders around your content.

WHY IT MATTERS

Most "why is my layout broken" moments come down to misunderstanding the box model — an element being bigger or smaller than you expected.

The core concept

The Box Model Diagram



Margin

```
margin: 20px;
```

Space OUTSIDE the border — pushes other elements away.

Border

```
border: 2px solid #333;
```

The visible edge, drawn around the padding.

Padding

```
padding: 16px;
```

Space INSIDE the border, around the content.

Content

```
width / height
```

Where your text, images, or children actually live.

Box model component 1 of 4

Content

The innermost area — your actual text, image, or child elements. This is what width and height control by default.

```
.card {  
  width: 300px;  
  height: 150px;  
}
```

preview



content
300 × 150

Box model component 2 of 4

Padding

padding is the space INSIDE the border, surrounding the content. It pushes the border outward without changing the content itself.

SHORTHAND ORDER

`padding: 10px 20px;` means 10px top & bottom, 20px left & right. Four values go top, right, bottom, left (clockwise).

preview

`padding: 30px 50px;`

content

Border

border is a visible (or invisible) line drawn around the padding. It has three parts you can set together or separately: width, style, and color.

```
.card {  
  border: 3px solid #D98324;  
}
```

```
/* or separately: */  
border-width: 3px;  
border-style: solid;  
border-color: #D98324;
```

preview

content + border



Box model component 4 of 4

Margin

margin is the space OUTSIDE the border — it's invisible, and it pushes other elements away from this one. It's the only layer that isn't part of the element's own visible box.

HANDY TRICK

`margin: 0 auto;` on a block element with a set width centers it horizontally — one of the most common CSS one-liners.

preview — two boxes pushed apart by margin



```
margin-right:  
40px;
```

box-sizing

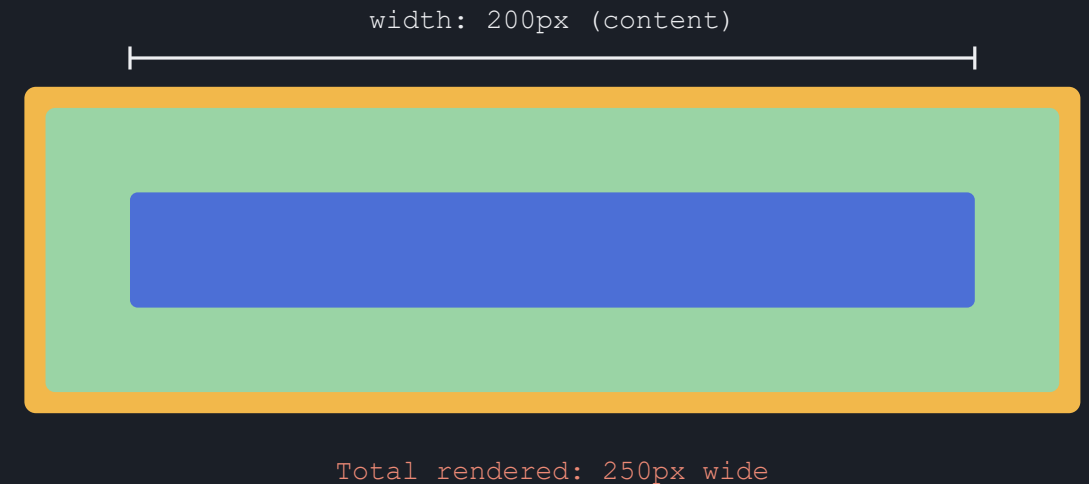
box-sizing: content-box (the default)

By default, width and height set the CONTENT area only. Padding and border get added ON TOP — so the final rendered box ends up bigger than the width you set.

```
.card {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
  /* box-sizing: content-box; is the default */  
}
```

THE MATH

Total rendered width = 200 (content) + 20+20 (padding) + 5+5 (border) = 250px. Not the 200px you might expect!



box-sizing

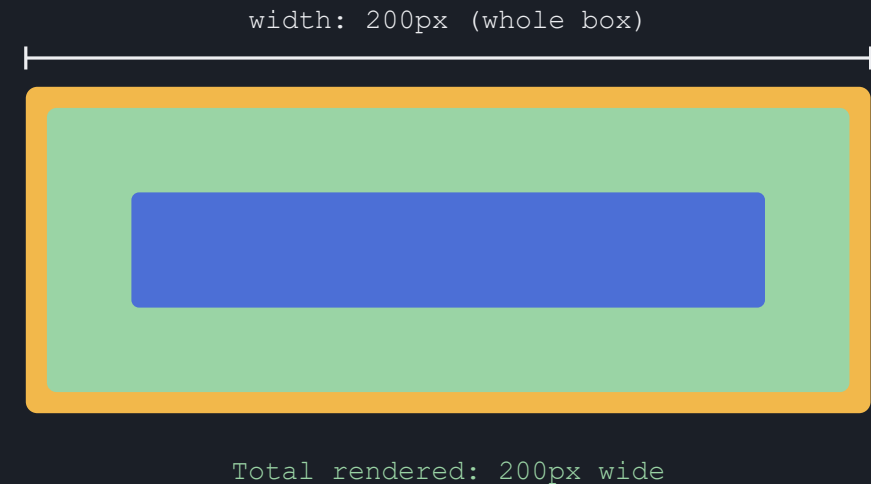
box-sizing: border-box

With border-box, width and height include the padding AND border. The content area shrinks to make room — but the total box stays exactly the size you set.

```
.card {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}
```

WHY MOST DEVELOPERS PREFER THIS

What you set is what you get — the declared width never silently grows. Far more predictable for building layouts.



Side by side

Same CSS, Two Different Final Sizes

content-box (default)



= 250px total

border-box



= 200px total

Both set width: 200px; padding: 20px; border: 5px solid; — only box-sizing differs.

THE UNIVERSAL RESET

`*, *::before, *::after { box-sizing: border-box; }` — almost every real project adds this at the very top of its CSS, once, for the whole page.

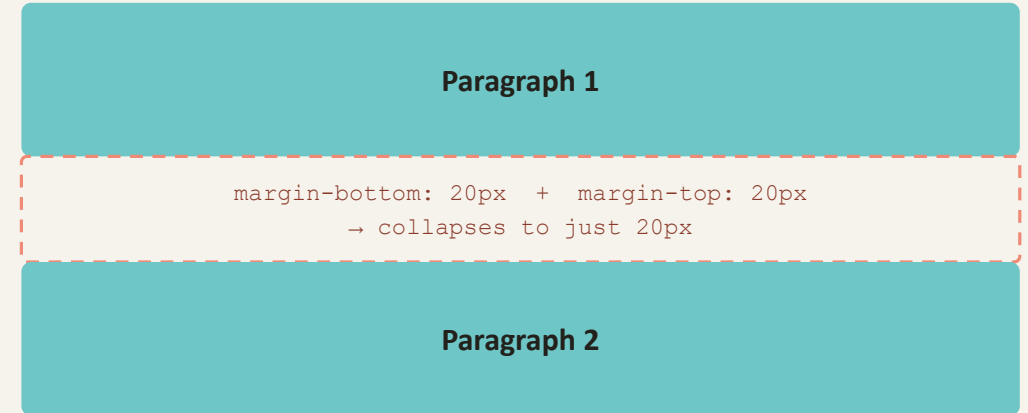
A common surprise

Margin Collapsing

When two block elements stack vertically, their touching top/bottom margins don't ADD UP — the browser collapses them into a single margin, equal to the LARGER of the two.

```
p {  
  margin-top: 20px;  
  margin-bottom: 20px;  
}
```

preview



Actual gap: 20px — not 40px!

A common surprise

When Does Margin Collapsing Happen?

- Only VERTICAL margins (top/bottom) collapse — left/right margins never do
- Only between block elements in normal document flow
- Only adjacent margins — if padding or a border sits between them, they won't collapse
- Flex and grid children never collapse their margins with each other

PRACTICAL TAKEAWAY

Don't assume two 20px margins stack into 40px of space. If your layout has less gap than expected, margin collapsing is often why.

The display property

Every Element Has a Default Display

display decides how an element behaves in the page's layout — whether it starts a new line, how its width works, and how the box model even applies to it.

block

`<div>, <p>, <h1>`

Starts on its own line, fills the available width.

inline

`<span>, <a>, <strong>`

Flows within a line of text, ignores width/height.

inline-block

set manually

Flows like inline, but respects width/height like block.

none

set manually

Removed from the page entirely — takes up no space.

Display values

display: block

A block element starts on a new line and stretches to fill the full width of its container by default. width, height, padding, and margin all work exactly as expected.

```
div {  
  display: block;  
}  
  
/* default for div, p, h1-h6,  
   section, header, footer... */
```

preview

Block 1 — full width

Block 2 — full width

Display values

display: inline

An inline element flows inside a line of text — it never starts a new line. width, height, and vertical margin are ignored; only horizontal spacing works.

```
span {  
  display: inline;  
}  
  
/* default for span, a,  
   strong, em, img... */
```

preview

This is a sentence with an **inline highlighted word** right in the middle, and the text keeps flowing normally around it without breaking to a new line.

[Display values](#)

display: inline-block

The best of both: it sits inline with surrounding content like inline, but you CAN set its width, height, padding, and margin, like block.

CLASSIC USE CASE

Navigation menu items or tag/badge chips — they sit side by side in a line, but each needs its own padding and size.

preview

Filed under:

HTML

CSS

Design

Display values

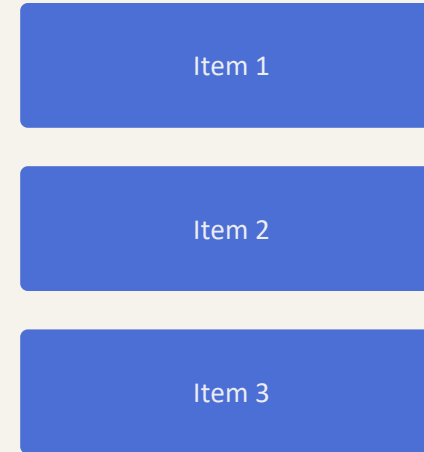
display: none

Completely removes the element from the page — no box, no space, as if it was never there. Other elements shift to fill the gap.

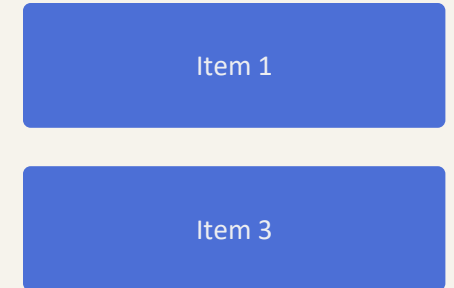
NOT THE SAME AS...

visibility: hidden; also hides the element, but leaves an empty gap behind — the space is still reserved.

before



after (item 2: none)



[Quick reference](#)

Display Values Cheat Sheet

Value	Starts new line?	width/height work?	Takes up space?
block	Yes	Yes	Yes
inline	No	No	Yes (text-sized)
inline-block	No	Yes	Yes
none	—	—	No — removed entirely

[Wrap-up](#)

Recap

You now know:

- The box model: content, padding, border, margin — in that order, inside out
- box-sizing: content-box adds padding/border on top of width; border-box keeps width as the total
- Vertical margins between block elements collapse to the larger one, not the sum
- display controls layout behavior: block, inline, inline-block, and none